

PROLOG

Practical Programs

Artificial Intelligence Laboratory
15 Complete Programs with Explanations & Sample Output

Topics Covered	
Family Trees & Relationships	List Operations (conc, reverse, member)
Arithmetic (sum, max, factorial)	Fibonacci Series & Power
List Aggregation (sum, max)	Insert & Delete by Position
Even/Odd Length Predicates	Recursive & Accumulator Patterns

Table of Contents

01. Family Tree and Family Relationships
02. List Concatenation — `conc(L1, L2, L3)`
03. List Reversal — `reverse(L, R)`
04. Sum of Two Numbers
05. Maximum of Two Numbers — `max(X, Y, M)`
06. Factorial — `factorial(N, F)`
07. Fibonacci Series — `generate_fib(N, T)`
08. Power — `power(Num, Pow, Ans)`
09. Multiplication — `multi(N1, N2, R)`
10. List Membership — `memb(X, L)`
11. Sum of List — `sumlist(L, S)`
12. Even and Odd Length Lists
13. Maximum in a List — `maxlist(L, M)`
14. Insert at Nth Position — `insert(I, N, L, R)`
15. Delete Nth Element — `delete(N, L, R)`

Program 01: Family Tree and Family Relationships

AIM:

Write a PROLOG program to implement the family tree and demonstrate the family relationship.

PROLOG CODE:

```
% Facts: parent(Parent, Child)
parent(tom, bob).
parent(tom, liz).
parent(bob, ann).
parent(bob, pat).
parent(pat, jim).

% Facts: male/female
male(tom). male(bob). male(pat). male(jim).
female(liz). female(ann).

% Rules
father(X, Y) :- parent(X, Y), male(X).
mother(X, Y) :- parent(X, Y), female(X).
grandparent(X, Z) :- parent(X, Y), parent(Y, Z).
sibling(X, Y) :- parent(Z, X), parent(Z, Y), X \= Y.
ancestor(X, Y) :- parent(X, Y).
ancestor(X, Y) :- parent(X, Z), ancestor(Z, Y).

% Queries
?- father(tom, bob). % true
?- grandparent(tom, ann). % true
?- sibling(bob, liz). % true
?- ancestor(tom, jim). % true
```

EXPLANATION:

1. Facts — parent/2 establishes the parent-child relationships. male/1 and female/1 tag each individual's gender.
2. father/2 and mother/2 — derived by combining the parent fact with the appropriate gender fact.
3. grandparent/2 — chains two parent calls: X is a grandparent of Z if X is a parent of Y and Y is a parent of Z.
4. sibling/2 — two people are siblings when they share a common parent and are not the same person (X \= Y prevents self-matching).
5. ancestor/2 — uses recursion: the base case is a direct parent; the recursive case walks further up the tree.

SAMPLE OUTPUT:

```
?- father(tom, bob).  
true.
```

```
?- grandparent(tom, ann).  
true.
```

```
?- sibling(bob, liz).  
true.
```

```
?- ancestor(tom, jim).  
true.
```

Program 02: List Concatenation — conc(L1, L2, L3)

AIM:

Write a PROLOG program to implement conc(L1, L2, L3) where L2 is appended with L1 to get L3.

PROLOG CODE:

```
% Base case: concatenating empty list with L2 gives L2
conc([], L2, L2).

% Recursive case
conc([H|T], L2, [H|T3]) :-
    conc(T, L2, T3).

% Query
?- conc([1,2,3], [4,5,6], L3).
% L3 = [1, 2, 3, 4, 5, 6]
```

EXPLANATION:

1. Base case — when L1 is empty (`[]`), the result L3 is simply L2. This is the termination condition.
2. Recursive case — the head H of L1 becomes the head of L3. The tail T of L1 is recursively concatenated with L2 to produce T3 (the tail of L3).
3. Unification drives the logic: PROLOG pattern-matches `[H|T]` against L1, pops H into the result, and recurses on the shrinking tail.
4. This predicate also works in reverse: `conc(X, Y, [1,2,3])` will enumerate all ways to split `[1,2,3]`.

SAMPLE OUTPUT:

```
?- conc([1,2,3],[4,5,6],L3).
L3 = [1, 2, 3, 4, 5, 6].
```

Program 03: List Reversal — reverse(L, R)

AIM:

Write a PROLOG program to implement reverse(L, R) where L is the original list and R is the reversed list.

PROLOG CODE:

```
% Accumulator-based reverse
rev([], Acc, Acc).
rev([H|T], Acc, R) :-
    rev(T, [H|Acc], R).

% User-facing wrapper
reverse_list(L, R) :- rev(L, [], R).

% Query
?- reverse_list([1,2,3,4,5], R).
% R = [5, 4, 3, 2, 1]
```

EXPLANATION:

1. Accumulator pattern — instead of building the reversed list after each recursive call returns, we prepend each element to an accumulator as we traverse forward.
2. rev/3 base case — when the input list is empty, the accumulator holds the reversed result, so we unify Acc with R.
3. rev/3 recursive case — the head H is prepended to the accumulator [H|Acc], effectively reversing the order, and the tail T is processed next.
4. reverse_list/2 is a clean wrapper that starts the accumulator as an empty list [].

SAMPLE OUTPUT:

```
?- reverse_list([1,2,3,4,5], R).
R = [5, 4, 3, 2, 1].
```

Program 04: Sum of Two Numbers

AIM:

Write a PROLOG program to calculate the sum of two numbers.

PROLOG CODE:

```
% Using the built-in 'is' operator for arithmetic evaluation
sum(X, Y, S) :-
    S is X + Y.

% Query
?- sum(10, 25, S).
% S = 35

?- sum(3.5, 2.5, S).
% S = 6.0
```

EXPLANATION:

1. The `is/2` operator tells PROLOG to arithmetically evaluate the right-hand side expression and unify the result with `S`.
2. Without `is`, writing `S = X + Y` would only create a symbolic term `+(X,Y)`, not compute the numeric value.
3. PROLOG handles both integers and floating-point numbers transparently through this mechanism.
4. `X` and `Y` must be instantiated (bound to values) before calling `sum/3`; otherwise PROLOG raises an instantiation error.

SAMPLE OUTPUT:

```
?- sum(10, 25, S).
S = 35.

?- sum(3.5, 2.5, S).
S = 6.0.
```

Program 05: Maximum of Two Numbers — max(X, Y, M)

AIM:

Write a PROLOG program to implement max(X, Y, M) so that M is the maximum of two numbers X and Y.

PROLOG CODE:

```
% Rule 1: X is the maximum when X >= Y
max(X, Y, X) :- X >= Y.

% Rule 2: Y is the maximum when Y > X
max(X, Y, Y) :- Y > X.

% Query
?- max(10, 20, M).
% M = 20

?- max(50, 30, M).
% M = 50

?- max(7, 7, M).
% M = 7
```

EXPLANATION:

1. Rule 1 — if $X \geq Y$, PROLOG unifies M with X (the third argument) and succeeds. No arithmetic assignment is needed because we pattern-match $M = X$ in the head.
2. Rule 2 — if $Y > X$ (strictly greater), M is unified with Y.
3. The two rules are mutually exclusive for distinct values. When $X == Y$, only Rule 1 fires ($\geq$ covers equality).
4. PROLOG evaluates $\geq$ and $>$ as built-in arithmetic comparisons via the internal is-evaluation mechanism.

SAMPLE OUTPUT:

```
?- max(10, 20, M).
M = 20.

?- max(50, 30, M).
M = 50.

?- max(7, 7, M).
M = 7.
```


Program 06: Factorial — factorial(N, F)

AIM:

Write a program in PROLOG to implement factorial(N, F) where F represents the factorial of a number N.

PROLOG CODE:

```
% Base case: 0! = 1
factorial(0, 1).

% Recursive case: N! = N * (N-1)!
factorial(N, F) :-
    N > 0,
    N1 is N - 1,
    factorial(N1, F1),
    F is N * F1.

% Query
?- factorial(5, F).
% F = 120

?- factorial(0, F).
% F = 1
```

EXPLANATION:

1. Base case — factorial(0, 1) defines $0! = 1$, which stops the recursion.
2. Guard condition — $N > 0$ prevents the recursive rule from firing for negative numbers, avoiding infinite loops.
3. $N1$ is $N - 1$ computes the decremented value. Without is, PROLOG would treat $N - 1$ as a symbolic expression.
4. The recursive call factorial(N1, F1) obtains $(N-1)!$, and then F is $N * F1$ computes the final product.
5. PROLOG uses a depth-first call stack here: it recurses all the way to factorial(0,1), then multiplies back up.

SAMPLE OUTPUT:

```
?- factorial(5, F).
F = 120.

?- factorial(0, F).
F = 1.
```

Program 07: Fibonacci Series — generate_fib(N, T)

AIM:

Write a program in PROLOG to implement generate_fib(N, T) where T represents the Nth term of the Fibonacci series.

PROLOG CODE:

```
% Base cases
generate_fib(0, 0).
generate_fib(1, 1).

% Recursive case: fib(N) = fib(N-1) + fib(N-2)
generate_fib(N, T) :-
    N > 1,
    N1 is N - 1,
    N2 is N - 2,
    generate_fib(N1, T1),
    generate_fib(N2, T2),
    T is T1 + T2.

% Query
?- generate_fib(7, T).
% T = 13

?- generate_fib(10, T).
% T = 55
```

EXPLANATION:

1. Base cases — $\text{fib}(0) = 0$ and $\text{fib}(1) = 1$ define the seed values of the Fibonacci sequence.
2. Guard $N > 1$ ensures the recursive rule only applies for positions 2 and above.
3. Two sub-problems — $N1$ and $N2$ store $N-1$ and $N-2$; the predicate is called for each, returning $T1$ and $T2$.
4. T is $T1 + T2$ sums the two preceding Fibonacci values to produce the Nth term.
5. Note: this naive recursive approach recalculates sub-problems repeatedly. For large N , an accumulator-based or memoized version is preferred.

SAMPLE OUTPUT:

```
?- generate_fib(7, T).
T = 13.

?- generate_fib(10, T).
T = 55.
```

Program 08: Power — power(Num, Pow, Ans)

AIM:

Write a PROLOG program to implement power(Num, Pow, Ans) where Num is raised to the power Pow to get Ans.

PROLOG CODE:

```
% Base case: any number raised to power 0 is 1
power(_, 0, 1).

% Recursive case: Num^Pow = Num * Num^(Pow-1)
power(Num, Pow, Ans) :-
    Pow > 0,
    Pow1 is Pow - 1,
    power(Num, Pow1, Ans1),
    Ans is Num * Ans1.

% Query
?- power(2, 10, Ans).
% Ans = 1024

?- power(3, 4, Ans).
% Ans = 81
```

EXPLANATION:

1. Base case — any base raised to the exponent 0 equals 1 (the identity for multiplication). The `_` wildcard means we don't care about the base value here.
2. Guard `Pow > 0` prevents unintended matching of the recursive rule for zero.
3. `Pow1 is Pow - 1` decrements the exponent, and the recursive call computes $\text{Num}^{(\text{Pow}-1)}$ into `Ans1`.
4. `Ans is Num * Ans1` multiplies the base once more to build up the final power.
5. This approach uses $O(\text{Pow})$ recursive calls. PROLOG's built-in `**` operator can also be used: `Ans is Num ** Pow`.

SAMPLE OUTPUT:

```
?- power(2, 10, Ans).
Ans = 1024.

?- power(3, 4, Ans).
Ans = 81.
```

Program 09: Multiplication — multi(N1, N2, R)

AIM:

Write a PROLOG program to implement multi(N1, N2, R) where N1 and N2 are the numbers to multiply and R is the result.

PROLOG CODE:

```
% Using built-in arithmetic
multi(N1, N2, R) :-
R is N1 * N2.

% Recursive definition (without built-in *)
multi_rec(_, 0, 0).
multi_rec(N1, N2, R) :-
N2 > 0,
N2_1 is N2 - 1,
multi_rec(N1, N2_1, R1),
R is R1 + N1.

% Query
?- multi(6, 7, R).
% R = 42

?- multi_rec(5, 4, R).
% R = 20
```

EXPLANATION:

1. Arithmetic version — R is $N1 * N2$ directly evaluates the product using PROLOG's built-in multiplication.
2. Recursive version — models multiplication as repeated addition: $N1 * N2 = N1 + N1 * (N2-1)$.
3. multi_rec base case — multiplying anything by 0 gives 0, halting the recursion.
4. multi_rec recursive case — decrements N2 by 1, recursively obtains $R1 = N1 * (N2-1)$, then adds N1 once to get the full product.
5. The arithmetic version is preferred in practice; the recursive form illustrates how multiplication can be defined from first principles.

SAMPLE OUTPUT:

```
?- multi(6, 7, R).
R = 42.

?- multi_rec(5, 4, R).
R = 20.
```

Program 10: List Membership — memb(X, L)

AIM:

Write a PROLOG program to implement memb(X, L) to check whether X is a member of list L or not.

PROLOG CODE:

```
% Base case: X is a member if it is the head of the list
memb(X, [X|_]).

% Recursive case: X is a member if it is in the tail
memb(X, [_|T]) :-
    memb(X, T).

% Query
?- memb(3, [1,2,3,4,5]).
% true

?- memb(6, [1,2,3,4,5]).
% false
```

EXPLANATION:

1. Base case — memb(X, [X|_]) succeeds immediately when X matches the head of the list. The tail is irrelevant (wildcard _).
2. Recursive case — if X is not the head, we ignore the head (wildcard _) and search the tail T.
3. PROLOG backtracks through all clauses: if the first clause fails (X != head), it tries the second clause on the tail.
4. When the list is exhausted ([]), neither clause matches, so PROLOG returns false — X is not in the list.
5. This predicate can also be used to generate members: memb(X, [1,2,3]) will enumerate X = 1, 2, 3 on backtracking.

SAMPLE OUTPUT:

```
?- memb(3, [1,2,3,4,5]).
true.

?- memb(6, [1,2,3,4,5]).
false.
```

Program 11: Sum of List — `sumlist(L, S)`

AIM:

Write a PROLOG program to implement `sumlist(L, S)` so that `S` is the sum of a given list `L`.

PROLOG CODE:

```
% Base case: sum of empty list is 0
sumlist([], 0).

% Recursive case
sumlist([H|T], S) :-
    sumlist(T, S1),
    S is S1 + H.

% Query
?- sumlist([1,2,3,4,5], S).
% S = 15

?- sumlist([], S).
% S = 0
```

EXPLANATION:

1. Base case — the sum of an empty list is defined as 0. This is both mathematically correct and serves as the recursion terminator.
2. Recursive case — the head `H` is extracted from the list. The predicate recurses to find `S1`, the sum of the tail `T`.
3. `S is S1 + H` adds the head to the tail-sum. This happens after the recursive call returns (post-order accumulation).
4. PROLOG walks to the end of the list first, then 'bubbles up' the additions as each call returns.

SAMPLE OUTPUT:

```
?- sumlist([1,2,3,4,5], S).
S = 15.

?- sumlist([], S).
S = 0.
```

Program 12: Even and Odd Length Lists

AIM:

Write a PROLOG program to implement `evenlength(List)` and `oddlength(List)` predicates.

PROLOG CODE:

```
% evenlength: true if the list has even number of elements
evenlength([]).
evenlength([_,_|T]) :-
    evenlength(T).

% oddlength: true if the list has odd number of elements
oddlength([_]).
oddlength([_,_|T]) :-
    oddlength(T).

% Query
?- evenlength([1,2,3,4]).
% true

?- oddlength([1,2,3]).
% true

?- evenlength([1,2,3]).
% false
```

EXPLANATION:

1. `evenlength` base case — an empty list `[]` has 0 elements, which is even, so the predicate succeeds.
2. `evenlength` recursive case — the pattern `[_,_|T]` strips two elements at a time. If we can keep removing pairs until the list is empty, it is even-length.
3. `oddlength` base case — a single-element list `[_]` has 1 element (odd), so it succeeds.
4. `oddlength` recursive case — also strips two elements at a time; if we reduce to a single element, the original count was odd.
5. Both predicates work by consuming pairs: even-length lists collapse to `[]`, odd-length lists collapse to `[_]`.

SAMPLE OUTPUT:

```
?- evenlength([1,2,3,4]).  
true.
```

```
?- oddlength([1,2,3]).  
true.
```

```
?- evenlength([1,2,3]).  
false.
```

Program 13: Maximum in a List — maxlist(L, M)

AIM:

Write a PROLOG program to implement maxlist(L, M) so that M is the maximum number in the list.

PROLOG CODE:

```
% Base case: max of single-element list is that element
maxlist([X], X).

% Recursive case
maxlist([H|T], M) :-
    maxlist(T, M1),
    (H > M1 -> M = H ; M = M1).

% Query
?- maxlist([3, 1, 4, 1, 5, 9, 2, 6], M).
% M = 9

?- maxlist([7], M).
% M = 7
```

EXPLANATION:

1. Base case — a list with one element: the maximum is that element itself.
2. Recursive case — we find M1 (the maximum of the tail T), then compare it with the head H.
3. If-then-else (-> ;) — if H > M1 then M = H (head is larger), otherwise M = M1 (tail's max wins).
4. The if-then-else construct (Cond -> Then ; Else) is deterministic: once the condition is tested, PROLOG commits to one branch without backtracking.
5. This approach makes O(n) comparisons in a single pass through the list.

SAMPLE OUTPUT:

```
?- maxlist([3,1,4,1,5,9,2,6], M).
M = 9.

?- maxlist([7], M).
M = 7.
```

Program 14: Insert at Nth Position — insert(I, N, L, R)

AIM:

Write a PROLOG program to implement insert(I, N, L, R) that inserts item I into Nth position of list L to generate list R.

PROLOG CODE:

```
% Insert at position 1: prepend I to the list
insert(I, 1, L, [I|L]).

% Insert at position N (N > 1): keep head, recurse on tail
insert(I, N, [H|T], [H|R]) :-
    N > 1,
    N1 is N - 1,
    insert(I, N1, T, R).

% Query
?- insert(99, 3, [1,2,3,4,5], R).
% R = [1, 2, 99, 3, 4, 5]

?- insert(0, 1, [1,2,3], R).
% R = [0, 1, 2, 3]
```

EXPLANATION:

1. Base case — when $N = 1$, we insert I at the front of the remaining list L, producing $[I|L]$.
2. Recursive case — while $N > 1$, we keep the current head H unchanged in the result $[H|R]$, decrement N to N1, and recurse into the tail T.
3. Each recursive call moves one position deeper into the list and decrements the counter.
4. When the counter finally reaches 1, the base case fires and I is inserted at that position.
5. Positions are 1-indexed: position 1 means before the first element, position 2 means before the second, and so on.

SAMPLE OUTPUT:

```
?- insert(99, 3, [1,2,3,4,5], R).
R = [1, 2, 99, 3, 4, 5].

?- insert(0, 1, [1,2,3], R).
R = [0, 1, 2, 3].
```

Program 15: Delete Nth Element — delete(N, L, R)

AIM:

Write a PROLOG program to implement delete(N, L, R) that removes the element at Nth position from list L to generate list R.

PROLOG CODE:

```
% Base case: delete element at position 1 (remove head)
delete(1, [_|T], T).

% Recursive case: keep head, delete from tail
delete(N, [H|T], [H|R]) :-
    N > 1,
    N1 is N - 1,
    delete(N1, T, R).

% Query
?- delete(3, [1,2,3,4,5], R).
% R = [1, 2, 4, 5]

?- delete(1, [10,20,30], R).
% R = [20, 30]
```

EXPLANATION:

1. Base case — delete(1, [_|T], T): when N = 1 we discard the head (using _ wildcard) and the result is simply the tail T.
2. Recursive case — while N > 1, the current head H is kept in the result [H|R], N is decremented, and we recurse into the tail T.
3. The logic mirrors the insert predicate but instead of adding an element when the counter reaches 1, we skip (discard) it.
4. Positions are 1-indexed. Attempting to delete beyond the list length will cause the predicate to fail.
5. This predicate is deterministic for valid inputs — it finds exactly one solution.

SAMPLE OUTPUT:

```
?- delete(3, [1,2,3,4,5], R).
R = [1, 2, 4, 5].

?- delete(1, [10,20,30], R).
R = [20, 30].
```

PROLOG

Additional Practice Programs

Artificial Intelligence Laboratory
11 Practice Programs — Number Theory, Graphs & More

Category	Programs
Number Theory	GCD, LCM, Power of 2, Sum/Count Digits
Sequences	Fibonacci List, Sum of N Naturals
Number Manipulation	Reverse Number, Palindrome Check
Graph Theory	Directed Graph Connectivity
Primality	Prime Number Test

Note: These programs are additional practice questions. They are not part of the core practical list but are highly useful for strengthening your understanding of PROLOG's recursive and arithmetic capabilities.

Table of Contents

01. GCD of Two Numbers — gcd(N1, N2, R)
02. LCM of Two Numbers — lcm(N1, N2, R)
03. Check Power of 2 — isPower2(N)
04. First N Fibonacci Terms — fibo(N, R)
05. Graph Connectivity — connected(Node1, Node2)
06. Sum of Digits — sumDigits(N, Result)
07. Count Digits — countDigits(N, Result)
08. Sum of First N Natural Numbers — sumN(N, Result)
09. Reverse a Number — rev(N, Result)
10. Palindrome Check — isPalin(N)
11. Prime Number Check — isPrime(N)

Practice 01: GCD of Two Numbers — gcd(N1, N2, R)

AIM:

Compute the GCD of two numbers using gcd(N1, N2, R), where N1 and N2 are the given numbers and R represents the greatest common divisor.

PROLOG CODE:

```
% Base case: gcd(N, 0, N) — when second number is 0, GCD is the first
gcd(N, 0, N) :- N > 0.

% Recursive case: Euclidean algorithm
gcd(N1, N2, R) :-
    N2 > 0,
    Rem is N1 mod N2,
    gcd(N2, Rem, R).

% Queries
?- gcd(48, 18, R). % R = 6
?- gcd(100, 75, R). % R = 25
?- gcd(17, 5, R). % R = 1
```

EXPLANATION:

1. Euclidean Algorithm — the GCD of two numbers equals the GCD of the smaller number and the remainder of dividing the larger by the smaller. This is one of the oldest known algorithms.
2. Base case — gcd(N, 0, N): when the second argument reaches 0, the first argument is the GCD. Guard $N > 0$ ensures validity.
3. Recursive case — Rem is $N1 \bmod N2$ computes the remainder. We then call gcd(N2, Rem, R), swapping the arguments per Euclid's method.
4. Each call reduces the problem: N2 shrinks on every iteration, guaranteeing termination.
5. For coprime numbers (e.g., 17 and 5), the GCD is 1, which is correctly returned when Rem eventually reaches 0.

SAMPLE OUTPUT:

```
?- gcd(48, 18, R).
R = 6.

?- gcd(100, 75, R).
R = 25.

?- gcd(17, 5, R).
R = 1.
```

Practice 02: LCM of Two Numbers — lcm(N1, N2, R)

AIM:

Compute the LCM of two numbers using lcm(N1, N2, R), where N1 and N2 are the given numbers and R represents the least common multiple.

PROLOG CODE:

```
% GCD helper (Euclidean)
gcd(N, 0, N) :- N > 0.
gcd(N1, N2, R) :-
    N2 > 0,
    Rem is N1 mod N2,
    gcd(N2, Rem, R).

% LCM using the relation: LCM(a,b) = (a * b) / GCD(a,b)
lcm(N1, N2, R) :-
    gcd(N1, N2, G),
    R is (N1 * N2) // G.

% Queries
?- lcm(4, 6, R). % R = 12
?- lcm(12, 18, R). % R = 36
?- lcm(7, 5, R). % R = 35
```

EXPLANATION:

1. Mathematical relationship — $\text{LCM}(a, b) = (a \times b) / \text{GCD}(a, b)$. This avoids iterating through multiples by reusing the GCD.
2. The gcd/3 helper from Program 1 is included here. In a real Prolog file you would define it once and reuse it.
3. R is $(N1 * N2) // G$ uses integer division (//) to ensure the result is always a whole number (the product is always exactly divisible by the GCD).
4. For coprime numbers ($\text{GCD} = 1$), LCM equals the product $N1 * N2$, which is the expected result.
5. This single-rule approach is clean and efficient — the heavy lifting is delegated to the GCD predicate.

SAMPLE OUTPUT:

```
?- lcm(4, 6, R).
R = 12.

?- lcm(12, 18, R).
R = 36.

?- lcm(7, 5, R).
R = 35.
```


Practice 03: Check Power of 2 — isPower2(N)

AIM:

Check whether a given number is a power of 2 using isPower2(N). The program should succeed if N is a power of 2 and fail otherwise.

PROLOG CODE:

```
% Base case: 1 = 2^0, so 1 is a power of 2
isPower2(1).

% Recursive case: N is a power of 2 if N is even and N/2 is also a power of 2
isPower2(N) :-
    N > 1,
    N mod 2 == 0,
    N1 is N // 2,
    isPower2(N1).

% Queries
?- isPower2(16). % true (2^4)
?- isPower2(32). % true (2^5)
?- isPower2(12). % false
?- isPower2(1). % true (2^0)
```

EXPLANATION:

1. Base case — isPower2(1) succeeds because $1 = 2^0$, the starting point of all powers of 2.
2. Guard $N > 1$ ensures the recursive rule does not match 0 or negative numbers.
3. $N \bmod 2 == 0$ checks that N is even. Any power of 2 greater than 1 must be divisible by 2.
4. $N1 \text{ is } N // 2$ halves the number. If N was a power of 2, $N/2$ is also a power of 2 (one exponent lower).
5. The recursion continues halving until we reach 1 (success) or an odd number > 1 (failure).

SAMPLE OUTPUT:

```
?- isPower2(16).
true.

?- isPower2(32).
true.

?- isPower2(12).
false.

?- isPower2(1).
true.
```

Practice 04: First N Fibonacci Terms — fibo(N, R)

AIM:

Display the first N terms of the Fibonacci series using fibo(N, R), where N is the number of terms and R is the resulting Fibonacci sequence.

PROLOG CODE:

```
% Helper: generates Fibonacci list using accumulators
% fibo_helper(N, Prev, Curr, Acc, Result)
fibo_helper(0, _, _, Acc, Result) :-
    reverse(Acc, Result).
fibo_helper(N, Prev, Curr, Acc, Result) :-
    N > 0,
    Next is Prev + Curr,
    N1 is N - 1,
    fibo_helper(N1, Curr, Next, [Curr|Acc], Result).

% Main predicate: fibo(N, R)
fibo(0, []).
fibo(1, [0]).
fibo(N, R) :-
    N > 1,
    N1 is N - 1,
    fibo_helper(N1, 0, 1, [0], R).

% Queries
?- fibo(8, R).
% R = [0, 1, 1, 2, 3, 5, 8, 13]

?- fibo(5, R).
% R = [0, 1, 1, 2, 3]
```

EXPLANATION:

1. Unlike generate_fib from the earlier set (which finds a single term), fibo/2 builds a complete list of the first N Fibonacci numbers.
2. Accumulator pattern — fibo_helper carries Prev and Curr (the two most recent values) and builds the list in reverse in Acc.
3. Each step computes Next = Prev + Curr, decrements the counter, and prepends Curr to the accumulator.
4. When N reaches 0, the accumulator holds the list in reverse order. reverse(Acc, Result) corrects the order.
5. Base cases for fibo/2 handle N=0 (empty list) and N=1 ([0]) directly. For N > 1, the helper starts with Prev=0, Curr=1 and Acc=[0].

SAMPLE OUTPUT:

```
?- fibo(8, R).  
R = [0, 1, 1, 2, 3, 5, 8, 13].
```

```
?- fibo(5, R).  
R = [0, 1, 1, 2, 3].
```

Practice 05: Graph Connectivity — connected(Node1, Node2)

AIM:

Implement connected(Node1, Node2) for the graph: a->b, a->c, b->d, c->d, c->f, d->e, f->g, g->h, i->j. Determine whether two nodes are directly or indirectly connected.

PROLOG CODE:

```
% Graph edges (directed)
edge(a, b). edge(a, c). edge(b, d).
edge(c, d). edge(c, f). edge(d, e).
edge(f, g). edge(g, h). edge(i, j).

% connected/2 using visited list to avoid cycles
connected(X, Y) :- connected(X, Y, []).

connected(X, Y, _) :-
    edge(X, Y).
connected(X, Y, Visited) :-
    edge(X, Z),
    \+ member(Z, Visited),
    connected(Z, Y, [Z|Visited]).

% Queries
?- connected(a, e). % true (a->b->d->e or a->c->d->e)
?- connected(a, h). % true (a->c->f->g->h)
?- connected(i, j). % true (direct edge)
?- connected(a, j). % false (disconnected component)
?- connected(h, a). % false (directed graph)
```

EXPLANATION:

1. Facts — edge(X, Y) declares a directed edge from X to Y. Nine edges define the graph with two disconnected components: {a,b,c,d,e,f,g,h} and {i,j}.
2. Base case — connected(X, Y, _) succeeds immediately if there is a direct edge from X to Y.
3. Recursive case — if no direct edge exists, find an intermediate node Z reachable from X, ensure Z has not been visited, then check if Y is reachable from Z.
4. Visited list — the third argument accumulates nodes already explored. \+ member(Z, Visited) (negation-as-failure) prevents revisiting nodes, avoiding infinite loops in cyclic searches.
5. Wrapper connected/2 starts the search with an empty visited list [], providing a clean public interface.
6. Directionality — since only edge/2 facts are used (not their reverse), the predicate respects direction: connected(h, a) fails because there is no path back from h to a.

SAMPLE OUTPUT:

```
?- connected(a, e).  
true.
```

```
?- connected(a, h).  
true.
```

```
?- connected(i, j).  
true.
```

```
?- connected(a, j).  
false.
```

```
?- connected(h, a).  
false.
```

Practice 06: Sum of Digits — sumDigits(N, Result)

AIM:

Compute the sum of the digits of a number using sumDigits(N, Result), where N is the given number and Result is the sum of its digits.

PROLOG CODE:

```
% Base case: single digit number
sumDigits(N, N) :- N < 10, N >= 0.

% Recursive case: extract last digit and recurse
sumDigits(N, Result) :-
    N >= 10,
    LastDigit is N mod 10,
    Rest is N // 10,
    sumDigits(Rest, R1),
    Result is R1 + LastDigit.

% Queries
?- sumDigits(1234, Result). % Result = 10
?- sumDigits(9999, Result). % Result = 36
?- sumDigits(7, Result). % Result = 7
```

EXPLANATION:

1. Base case — a number with a single digit (0-9) is its own digit sum. $N < 10$ and $N \geq 0$ guards this case.
2. Digit extraction — $N \bmod 10$ isolates the last digit (the units place). $N // 10$ removes that digit, shifting all digits one place right.
3. Recursive step — sumDigits recurses on Rest (the number with the last digit removed), obtaining R1. The final result adds the extracted LastDigit to R1.
4. This processes digits from right to left (units first), but since addition is commutative the order does not affect the sum.
5. For a 4-digit number like 1234: extracts 4 -> recurses on 123 -> extracts 3 -> recurses on 12 -> ... until single digit 1 is reached.

SAMPLE OUTPUT:

```
?- sumDigits(1234, Result).  
Result = 10.
```

```
?- sumDigits(9999, Result).  
Result = 36.
```

```
?- sumDigits(7, Result).  
Result = 7.
```

Practice 07: Count Digits — countDigits(N, Result)

AIM:

Count the number of digits in a given number using countDigits(N, Result), where N is the input number and Result is the count of digits.

PROLOG CODE:

```
% Base case: single digit number has 1 digit
countDigits(N, 1) :- N < 10, N >= 0.

% Recursive case: remove last digit, count remaining, add 1
countDigits(N, Result) :-
    N >= 10,
    N1 is N // 10,
    countDigits(N1, R1),
    Result is R1 + 1.

% Queries
?- countDigits(12345, Result). % Result = 5
?- countDigits(9, Result). % Result = 1
?- countDigits(1000000, Result). % Result = 7
```

EXPLANATION:

1. Base case — any number from 0 to 9 has exactly 1 digit. The count is returned directly as 1.
2. Recursive case — $N // 10$ removes the last digit. We count the digits of the shorter number (R1), then add 1 to account for the digit we stripped.
3. This is structurally identical to sumDigits but instead of accumulating the digit's value, we accumulate a count of 1 per digit.
4. Each recursive call reduces N by one decimal place (divides by 10), so the depth of recursion equals the number of digits.
5. For 12345: strips 5 -> recurses on 1234 -> strips 4 -> ... -> reaches 1 (base: count=1) -> adds back 4 ones = 5 total.

SAMPLE OUTPUT:

```
?- countDigits(12345, Result).
Result = 5.

?- countDigits(9, Result).
Result = 1.

?- countDigits(1000000, Result).
Result = 7.
```


Practice 08: Sum of First N Natural Numbers — sumN(N, Result)

AIM:

Compute the sum of the first N natural numbers using sumN(N, Result), where N is the count and Result is their sum.

PROLOG CODE:

```
% Base case: sum of first 1 natural number is 1
sumN(1, 1).

% Recursive case: sum(N) = N + sum(N-1)
sumN(N, Result) :-
    N > 1,
    N1 is N - 1,
    sumN(N1, R1),
    Result is R1 + N.

% Alternative: direct formula (O(1))
sumN_formula(N, Result) :-
    Result is N * (N + 1) // 2.

% Queries
?- sumN(10, Result). % Result = 55
?- sumN(100, Result). % Result = 5050
?- sumN_formula(100, Result). % Result = 5050
```

EXPLANATION:

1. Base case — the sum of the first 1 natural number is 1. This anchors the recursion.
2. Recursive case — $\text{sumN}(N) = N + \text{sumN}(N-1)$. We decrement N, compute the sub-sum R1, then add the current N.
3. This mirrors the classic Gauss story: $1+2+3+\dots+N = N*(N+1)/2$.
4. sumN_formula/2 demonstrates the closed-form O(1) alternative using the Gauss formula, which avoids N recursive calls entirely.
5. Both versions give identical results. The recursive version illustrates PROLOG's inductive reasoning; the formula version shows how mathematical insight can replace computation.

SAMPLE OUTPUT:

```
?- sumN(10, Result).
```

```
Result = 55.
```

```
?- sumN(100, Result).
```

```
Result = 5050.
```

```
?- sumN_formula(100, Result).
```

```
Result = 5050.
```

Practice 09: Reverse a Number — rev(N, Result)

AIM:

Reverse a given number using rev(N, Result), where N is the input number and Result is the reversed number.

PROLOG CODE:

```
% Helper with accumulator
rev_helper(0, Acc, Acc).
rev_helper(N, Acc, Result) :-
    N > 0,
    Digit is N mod 10,
    NewAcc is Acc * 10 + Digit,
    N1 is N // 10,
    rev_helper(N1, NewAcc, Result).

% Main predicate
rev(N, Result) :- rev_helper(N, 0, Result).

% Queries
?- rev(12345, Result). % Result = 54321
?- rev(1200, Result). % Result = 21
?- rev(7, Result). % Result = 7
```

EXPLANATION:

1. Accumulator approach — instead of building the reversed number after returning from recursion, we build it incrementally as we peel off digits left-to-right.
2. Digit extraction — Digit is N mod 10 gets the last digit of N. N // 10 removes it.
3. Building the reverse — NewAcc is Acc * 10 + Digit shifts the accumulator left one decimal place and appends the new digit. This is the key step that constructs the reversed number.
4. Base case — rev_helper(0, Acc, Acc): when all digits are consumed, the accumulator holds the reversed number.
5. Note on trailing zeros — rev(1200, Result) gives 21 because leading zeros are dropped in numeric representation. This is mathematically correct behaviour.

SAMPLE OUTPUT:

```
?- rev(12345, Result).
Result = 54321.

?- rev(1200, Result).
Result = 21.

?- rev(7, Result).
Result = 7.
```


Practice 10: Palindrome Check — isPalin(N)

AIM:

Check whether a given number is a palindrome using isPalin(N). The program should succeed if N is a palindrome and fail otherwise.

PROLOG CODE:

```
% Reuse rev_helper from Program 9
rev_helper(0, Acc, Acc).
rev_helper(N, Acc, Result) :-
    N > 0,
    Digit is N mod 10,
    NewAcc is Acc * 10 + Digit,
    N1 is N // 10,
    rev_helper(N1, NewAcc, Result).

rev(N, Result) :- rev_helper(N, 0, Result).

% isPalin: a number is a palindrome if it equals its reverse
isPalin(N) :-
    N >= 0,
    rev(N, N). % reversed number must equal N itself

% Queries
?- isPalin(121). % true
?- isPalin(12321). % true
?- isPalin(123). % false
?- isPalin(7). % true (single digit)
```

EXPLANATION:

1. Core idea — a number is a palindrome if reversing it produces the same number. We reuse rev/2 from Program 9.
2. rev(N, N) — the second argument is N itself (not a fresh variable). PROLOG will reverse N and check if the result unifies with N. If they match, the call succeeds; otherwise it fails.
3. This is a beautiful example of PROLOG's unification: instead of comparing with `=:=`, we unify the reversed result directly with the input.
4. Single-digit numbers are always palindromes because rev(7, 7) is trivially true.
5. Numbers with trailing zeros (like 120) will not be palindromes because their reversal drops the leading zero (021 = 21 != 120).

SAMPLE OUTPUT:

```
?- isPalin(121).  
true.
```

```
?- isPalin(12321).  
true.
```

```
?- isPalin(123).  
false.
```

```
?- isPalin(7).  
true.
```

Practice 11: Prime Number Check — isPrime(N)

AIM:

Check whether a given number is a prime number using isPrime(N). The program should succeed if N is prime and fail otherwise.

PROLOG CODE:

```
% Helper: check no divisor exists in range [2, sqrt(N)]
has_divisor(N, D) :-
    D * D <= N,
    N mod D =:= 0.
has_divisor(N, D) :-
    D * D <= N,
    D1 is D + 1,
    has_divisor(N, D1).

% isPrime: N is prime if N > 1 and it has no divisors
isPrime(2).
isPrime(N) :-
    N > 2,
    N mod 2 =\= 0, % eliminate even numbers early
    \+ has_divisor(N, 3). % check odd divisors from 3

% Queries
?- isPrime(7). % true
?- isPrime(13). % true
?- isPrime(1). % false
?- isPrime(15). % false (3 x 5)
?- isPrime(97). % true
```

EXPLANATION:

1. has_divisor/2 — checks if any integer D in the range [startD, sqrt(N)] divides N. If $N \bmod D = 0$, a divisor is found.
2. Optimisation 1 — we only check up to sqrt(N) (the condition $D * D \leq N$). Any factor larger than the square root must pair with a factor smaller than it, so we would have found it already.
3. Optimisation 2 — $N \bmod 2 \neq 0$ quickly eliminates even numbers. Only odd divisors (starting from 3) are tested by has_divisor.
4. $\backslash +$ has_divisor(N, 3) — negation-as-failure: isPrime succeeds if no divisor can be found. If has_divisor ever succeeds, isPrime fails.
5. Special cases — isPrime(2) handles 2 explicitly (the only even prime). 1 is not prime because no clause matches $N=1$.

SAMPLE OUTPUT:

```
?- isPrime(7).  
true.
```

```
?- isPrime(13).  
true.
```

```
?- isPrime(1).  
false.
```

```
?- isPrime(15).  
false.
```

```
?- isPrime(97).  
true.
```
